

Code for Object-Oriented programming

```
//start
```

```
//Task1
```

```
#include <iostream>
```

```
#include <vector>
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <string>
```

```
#include <map>
```

```
#include <algorithm>
```

```
#include <limits>
```

```
#include <numeric>
```

```
#include <cmath>
```

```
#include <iomanip> // For std::setw
```

```
// Define a Candlestick class
```

```
class Candlestick {
```

```
public:
```

```
    double open;
```

```
    double close;
```

```
    double high;
```

```
    double low;
```

```
    int year;
```

```
// Constructor
```

```
Candlestick(int yr, double op, double cl, double hi, double lo)
```

```
    : year(yr), open(op), close(cl), high(hi), low(lo) {}
```

```
};
```

```
// Function to safely convert a string to double
```

```
double safeStringToDouble(const std::string& str) {  
    try {  
        if (str.empty()) throw std::invalid_argument("Empty string");  
        return std::stod(str);  
    } catch (const std::exception&) {  
        return std::numeric_limits<double>::quiet_NaN();  
    }  
}
```

```
// Function to compute candlestick data
```

```
std::vector<Candlestick> computeCandlestickData(const std::string& location, const  
std::string& filePath) {  
    std::vector<Candlestick> candlesticks;  
    std::ifstream file(filePath);  
  
    if (!file.is_open()) {  
        std::cerr << "Error: Unable to open file: " << filePath << std::endl;  
        return candlesticks;  
    }  
  
    std::string line;  
    std::map<std::string, int> locationColumnIndex;  
    std::vector<double> previousYearTemps;  
    std::vector<double> currentYearTemps;  
    int currentYear = 0;
```

```

double yearlyHigh = std::numeric_limits<double>::lowest();
double yearlyLow = std::numeric_limits<double>::max();

// Read the header line and map column indices
if (std::getline(file, line)) {
    std::stringstream ss(line);
    std::string columnName;
    int index = 0;
    while (std::getline(ss, columnName, ',')) {
        locationColumnIndex[columnName] = index++;
    }
}

// Ensure the specified location exists in the dataset
if (locationColumnIndex.find(location) == locationColumnIndex.end()) {
    std::cerr << "Error: Location " << location << " not found in the dataset.\n";
    return candlesticks;
}

int columnIndex = locationColumnIndex[location];

// Process each line of data
while (std::getline(file, line)) {
    std::stringstream ss(line);
    std::string date, temperatureStr;
    double temperature;
    int columnCounter = 0;

```

```

// Parse date
std::getline(ss, date, ',');

// Locate the correct temperature column
for (int i = 0; i <= columnIndex; ++i) {
    std::getline(ss, temperatureStr, ',');
}

temperature = safeStringToDouble(temperatureStr);
if (std::isnan(temperature)) continue;

// Extract the year from the date (format assumed: YYYY-MM-DD)
int year = std::stoi(date.substr(0, 4));

// Handle year change
if (currentYear != year) {
    if (currentYear != 0 && !currentYearTemps.empty()) {
        // Compute 'open' as the average temperature of the previous year
        double open = !previousYearTemps.empty() ?
            std::accumulate(previousYearTemps.begin(), previousYearTemps.end(),
0.0) / previousYearTemps.size() :
            std::numeric_limits<double>::quiet_NaN();

        // Compute 'close' as the average temperature of the current year
        double close = std::accumulate(currentYearTemps.begin(),
currentYearTemps.end(), 0.0) / currentYearTemps.size();

        // Add a new candlestick object for the completed year
        candlesticks.emplace_back(currentYear, open, close, yearlyHigh, yearlyLow);
    }
}

```

```

        // Transition to the new year: move current year's temperatures to the previous
year
        previousYearTemps = currentYearTemps;
        currentYearTemps.clear();

        // Reset yearly high and low temperatures
        currentYear = year;
        yearlyHigh = std::numeric_limits<double>::lowest();
        yearlyLow = std::numeric_limits<double>::max();
    }

    // Add the current temperature to the list and update yearly high/low
    currentYearTemps.push_back(temperature);
    yearlyHigh = std::max(yearlyHigh, temperature);
    yearlyLow = std::min(yearlyLow, temperature);
}

// Handle the last year in the dataset
if (!currentYearTemps.empty()) {
    double open = !previousYearTemps.empty() ?
        std::accumulate(previousYearTemps.begin(), previousYearTemps.end(), 0.0) /
previousYearTemps.size() :
        std::numeric_limits<double>::quiet_NaN();

    double close = std::accumulate(currentYearTemps.begin(), currentYearTemps.end(),
0.0) / currentYearTemps.size();

    candlesticks.emplace_back(currentYear, open, close, yearlyHigh, yearlyLow);
}

```

```

    return candlesticks;
}

// Helper function to print candlestick data
void printCandlestickData(const std::vector<Candlestick>& candlesticks) {
    std::cout << std::setw(8) << "Year" << std::setw(12) << "Open"
        << std::setw(12) << "Close" << std::setw(12) << "High"
        << std::setw(12) << "Low" << std::endl;
    std::cout << std::string(56, '-') << std::endl;

    for (const auto& c : candlesticks) {
        std::cout << std::setw(8) << c.year
            << std::setw(12) << std::fixed << std::setprecision(2) << c.open
            << std::setw(12) << c.close
            << std::setw(12) << c.high
            << std::setw(12) << c.low << std::endl;
    }
}

// Main function
int main() {
    std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";

    // Get the country code from the user
    std::string location;
    std::cout << "Enter countrycode_temperature (e.g., GB_temperature): ";
    std::cin >> location;

```

```
std::vector<Candlestick> candlestickData = computeCandlestickData(location,
filePath);
```

```
if (candlestickData.empty()) {
    std::cout << "No data available for the specified country." << std::endl;
    return 1;
}
```

```
printCandlestickData(candlestickData);
```

```
return 0;
}
```

//Task 2

```
#include <iostream>
```

```
#include <vector>
```

```
#include <fstream>
```

```
#include <sstream>
```

```
#include <string>
```

```
#include <algorithm>
```

```
#include <limits>
```

```
#include <numeric>
```

```
#include <cmath>
```

```
#include <map>
```

```
#include <iomanip>
```

```
// Define the Candlestick class
```

```

class Candlestick {
public:
    double open;
    double close;
    double high;
    double low;
    int year;

    // Constructor
    Candlestick(int yr, double op, double cl, double hi, double lo)
        : year(yr), open(op), close(cl), high(hi), low(lo) {}
};

// Function to safely convert a string to double
double safeStringToDouble(const std::string& str) {
    try {
        if (str.empty()) throw std::invalid_argument("Empty string");
        return std::stod(str);
    } catch (...) {
        return std::numeric_limits<double>::quiet_NaN();
    }
}

// Function to compute candlestick data from the CSV file
std::vector<Candlestick> computeCandlestickData(const std::string& location, const
std::string& filePath) {
    std::vector<Candlestick> candlesticks;
    std::ifstream file(filePath);

```



```

if (!file.is_open()) {
    std::cerr << "Error: Unable to open file: " << filePath << std::endl;
    return candlesticks;
}

std::string line;
std::map<std::string, int> locationColumnIndex;
std::vector<double> previousYearTemps, currentYearTemps;
int currentYear = 0;
double yearlyHigh = std::numeric_limits<double>::lowest();
double yearlyLow = std::numeric_limits<double>::max();

// Parse header row to get column indices
if (std::getline(file, line)) {
    std::stringstream ss(line);
    std::string columnName;
    int index = 0;
    while (std::getline(ss, columnName, ',')) {
        locationColumnIndex[columnName] = index++;
    }
}

if (locationColumnIndex.find(location) == locationColumnIndex.end()) {
    std::cerr << "Error: Location " << location << " not found in the dataset.\n";
    return candlesticks;
}

```

```

int columnIndex = locationColumnIndex[location];

// Read data rows
while (std::getline(file, line)) {
    std::stringstream ss(line);
    std::string date, temperatureStr;
    double temperature;

    std::getline(ss, date, ','); // Parse date

    // Parse the specified location's temperature column
    for (int i = 0; i <= columnIndex; ++i) {
        std::getline(ss, temperatureStr, ',');
    }

    temperature = safeStringToDouble(temperatureStr);
    if (std::isnan(temperature)) continue;

    int year = std::stoi(date.substr(0, 4));
    if (currentYear != year) {
        if (!currentYearTemps.empty()) {
            double open = previousYearTemps.empty() ?
std::numeric_limits<double>::quiet_NaN()
                : std::accumulate(previousYearTemps.begin(),
previousYearTemps.end(), 0.0) / previousYearTemps.size();

            double close = std::accumulate(currentYearTemps.begin(),
currentYearTemps.end(), 0.0) / currentYearTemps.size();

            candlesticks.emplace_back(currentYear, open, close, yearlyHigh, yearlyLow);
        }
    }
}

```

```

    previousYearTemps = currentYearTemps;

    currentYearTemps.clear();

    currentYear = year;

    yearlyHigh = std::numeric_limits<double>::lowest();
    yearlyLow = std::numeric_limits<double>::max();
}

    currentYearTemps.push_back(temperature);
    yearlyHigh = std::max(yearlyHigh, temperature);
    yearlyLow = std::min(yearlyLow, temperature);
}

// Process the final year
if (!currentYearTemps.empty()) {
    double open = previousYearTemps.empty() ?
std::numeric_limits<double>::quiet_NaN()
        : std::accumulate(previousYearTemps.begin(),
previousYearTemps.end(), 0.0) / previousYearTemps.size();

    double close = std::accumulate(currentYearTemps.begin(), currentYearTemps.end(),
0.0) / currentYearTemps.size();

    candlesticks.emplace_back(currentYear, open, close, yearlyHigh, yearlyLow);
}

return candlesticks;
}

// Function to print vertical candlestick plot with values
void printVerticalCandlestickPlot(const std::vector<Candlestick>& candlesticks) {
    const int plotHeight = 10;

```

```

for (const auto& c : candlesticks) {
    // Print the legend for this year's data

    std::cout << "Legend:\n";

    std::cout << " H - High temperature\n";

    std::cout << " O - Open (average temperature of the previous year)\n";

    std::cout << " C - Close (average temperature of the current year)\n";

    std::cout << " L - Low temperature\n";

    std::cout << std::string(30, '-') << "\n";


    std::cout << "Year " << c.year << ":\n";

    std::cout << " Open: " << c.open << " | Close: " << c.close
        << " | High: " << c.high << " | Low: " << c.low << "\n";


    double range = c.high - c.low;

    int openPos = static_cast<int>((c.open - c.low) / range * (plotHeight - 1));

    int closePos = static_cast<int>((c.close - c.low) / range * (plotHeight - 1));

    int highPos = 0;

    int lowPos = plotHeight - 1;


    if (openPos == closePos) {
        if (openPos < plotHeight - 1) {
            ++closePos;
        } else {
            --openPos;
        }
    }
}

```

```

// Adjust min and max temperatures to the nearest multiples of 10
int minTemp = static_cast<int>(std::floor(c.low / 10.0)) * 10;
int maxTemp = static_cast<int>(std::ceil(c.high / 10.0)) * 10;

// Generate temperature values for the scale
std::vector<int> tempScale;
for (int temp = maxTemp; temp >= minTemp; temp -= 10) {
    tempScale.push_back(temp);
}

// Map temperature values to their respective rows
std::map<int, int> tempRows;
for (size_t i = 0; i < tempScale.size(); ++i) {
    int row = static_cast<int>((c.high - tempScale[i]) / range * (plotHeight - 1));
    tempRows[row] = tempScale[i];
}

for (int i = 0; i < plotHeight; ++i) {
    // Print temperature value if it matches the row
    if (tempRows.find(i) != tempRows.end()) {
        std::cout << std::setw(4) << tempRows[i] << " ";
    } else {
        std::cout << "    "; // Empty space for alignment
    }

    // Display candlestick representation
    if (i == openPos) std::cout << " O ";
    else if (i == closePos) std::cout << " C ";
}

```

```

        else if (i == highPos) std::cout << " H ";
        else if (i == lowPos) std::cout << " L ";
        else std::cout << " | ";
        std::cout << '\n';
    }

    std::cout << '\n';
}

// Main function
int main() {
    std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";

    // Get the country code from the user
    std::string location;
    std::cout << "Enter the country code (e.g., GB_temperature): ";
    std::cin >> location;

    auto candlestickData = computeCandlestickData(location, filePath);

    if (candlestickData.empty()) {
        std::cerr << "No data available for the specified country." << std::endl;
        return 1;
    }

    printVerticalCandlestickPlot(candlestickData);

```

```
    return 0;
}
```

//Task 3

```
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <string>
#include <algorithm>
#include <limits>
#include <numeric>
#include <cmath>
#include <map>
#include <iomanip> // For std::setw
```

```
// Define the Candlestick class
```

```
class Candlestick {
```

```
public:
```

```
    double open;
```

```
    double close;
```

```
    double high;
```

```
    double low;
```

```
    int year;
```

```
// Constructor
```

```
Candlestick(int yr, double op, double cl, double hi, double lo)
```

```
    : year(yr), open(op), close(cl), high(hi), low(lo) {}
```

```
};
```

```
// Function to safely convert a string to double
```

```
double safeStringToDouble(const std::string& str) {  
    try {  
        if (str.empty()) throw std::invalid_argument("Empty string");  
        return std::stod(str);  
    } catch (...) {  
        return std::numeric_limits<double>::quiet_NaN();  
    }  
}
```

```
// Function to read and process the file for a specific country
```

```
std::vector<Candlestick> readAndProcessFile(const std::string& filePath, const  
std::string& country) {  
    std::ifstream file(filePath);  
    if (!file.is_open()) {  
        std::cerr << "Error: Unable to open file: " << filePath << std::endl;  
        return {};  
    }  
  
    std::string line;  
    std::vector<Candlestick> candlesticks;  
    std::map<int, std::vector<double>> yearlyTemps;  
  
    std::getline(file, line); // Read the header  
    std::stringstream headerStream(line);  
    std::string columnName;
```



```

int countryColumnIndex = -1, currentIndex = 0;

while (std::getline(headerStream, columnName, ',')) {
    if (columnName == country) {
        countryColumnIndex = currentIndex;
        break;
    }
    currentIndex++;
}

if (countryColumnIndex == -1) {
    std::cerr << "Error: Country not found in the dataset.\n";
    return {};
}

while (std::getline(file, line)) {
    std::stringstream lineStream(line);
    std::string date, tempStr;
    std::getline(lineStream, date, ','); // Extract date

    int year = std::stoi(date.substr(0, 4));

    for (int i = 0; i <= countryColumnIndex; ++i) {
        std::getline(lineStream, tempStr, ',');
    }

    double temperature = safeStringToDouble(tempStr);
    if (!std::isnan(temperature)) {

```

```

        yearlyTemps[year].push_back(temperature);
    }
}

std::vector<double> prevYearTemps;

for (const auto& pair : yearlyTemps) {
    int year = pair.first;
    const auto& temps = pair.second;

    if (!temps.empty()) {
        // Calculate average mean temperatures

        double currentYearAvg = std::accumulate(temps.begin(), temps.end(), 0.0) /
temps.size();

        double prevYearAvg = prevYearTemps.empty() ?
std::numeric_limits<double>::quiet_NaN()

        : std::accumulate(prevYearTemps.begin(),
prevYearTemps.end(), 0.0) / prevYearTemps.size();

        double high = *std::max_element(temps.begin(), temps.end());
        double low = *std::min_element(temps.begin(), temps.end());

        // Add the candlestick

        candlesticks.emplace_back(year, prevYearAvg, currentYearAvg, high, low);

        // Save current year temps for the next iteration

        prevYearTemps = temps;
    }
}

```

```

    return candlesticks;
}

// Function to filter candlestick data by date range
std::vector<Candlestick> filterByDateRange(const std::vector<Candlestick>& data, int
startYear, int endYear) {
    std::vector<Candlestick> filteredData;
    for (const auto& c : data) {
        if (c.year >= startYear && c.year <= endYear) {
            filteredData.push_back(c);
        }
    }
    return filteredData;
}

// Function to filter candlestick data by temperature range
std::vector<Candlestick> filterByTemperatureRange(const std::vector<Candlestick>&
data, double minTemp, double maxTemp) {
    std::vector<Candlestick> filteredData;
    for (const auto& c : data) {
        if (c.high <= maxTemp && c.low >= minTemp) {
            filteredData.push_back(c);
        }
    }
    return filteredData;
}

// Function to create a text-based graph

```

```

void plotTextGraph(const std::vector<Candlestick>& data, int plotHeight = 10) {
    if (data.empty()) {
        std::cout << "No data to plot.\n";
        return;
    }

    // Determine the global min and max temperatures for scaling
    double globalMin = std::numeric_limits<double>::max();
    double globalMax = std::numeric_limits<double>::lowest();

    for (const auto& c : data) {
        globalMin = std::min(globalMin, c.low);
        globalMax = std::max(globalMax, c.high);
    }

    // Adjust globalMin and globalMax to the nearest multiple of 5
    int roundedMin = static_cast<int>(std::floor(globalMin / 5.0)) * 5;
    int roundedMax = static_cast<int>(std::ceil(globalMax / 5.0)) * 5;

    // Ensure the tick interval is 5
    int tickInterval = 5;

    // Display temperature values above the graph
    std::cout << "\nTemperature Values:\n";
    for (const auto& c : data) {
        std::cout << "Year " << c.year << ": "
            << "Open: " << std::fixed << std::setprecision(1) << c.open << " | "
            << "Close: " << c.close << " | "

```

```

        << "High: " << c.high << " | "
        << "Low: " << c.low << "\n";
    }

    std::cout << "\n";

    // Create a grid to store characters at each row and column

    std::vector<std::vector<std::string>> grid(plotHeight + 1,
    std::vector<std::string>(data.size(), " "));

    // Populate the grid with H, C, O, L based on actual values
    for (size_t i = 0; i < data.size(); ++i) {
        const auto& c = data[i];

        // Calculate positions on the grid

        int highRow = static_cast<int>(std::round((c.high - roundedMin) / (roundedMax -
roundedMin) * plotHeight));

        int closeRow = static_cast<int>(std::round((c.close - roundedMin) / (roundedMax -
roundedMin) * plotHeight));

        int openRow = static_cast<int>(std::round((c.open - roundedMin) / (roundedMax -
roundedMin) * plotHeight));

        int lowRow = static_cast<int>(std::round((c.low - roundedMin) / (roundedMax -
roundedMin) * plotHeight));

        // Insert characters into the grid, ensuring no overlap
        if (grid[highRow][i] == " ") {
            grid[highRow][i] = " H ";
        }

        if (grid[closeRow][i] == " ") {
            grid[closeRow][i] = " C ";

```

```

    } else if (closeRow > 0 && c.close > c.high) {
        grid[closeRow - 1][i] = " C "; // Shift up
    }
    if (grid[openRow][i] == " ") {
        grid[openRow][i] = " O ";
    } else if (openRow > 0 && c.open > c.close) {
        grid[openRow - 1][i] = " O "; // Shift up
    }
    if (grid[lowRow][i] == " ") {
        grid[lowRow][i] = " L ";
    } else if (lowRow > 0 && c.low > c.open) {
        grid[lowRow - 1][i] = " L "; // Shift up
    }
}

// Generate the graph
for (int row = plotHeight; row >= 0; --row) {
    int currentValue = roundedMin + (row * tickInterval);

    std::cout << std::setw(6) << currentValue << " |";
    for (size_t col = 0; col < data.size(); ++col) {
        // Print centered characters in the grid
        std::cout << std::setw(6) << grid[row][col];
    }
    std::cout << "\n";

    // Add an empty row for spacing, except for the last row
    if (row > 0) {

```

```

std::cout << std::setw(6) << " " << "|";

for (size_t col = 0; col < data.size(); ++col) {

    std::cout << std::setw(6) << " ";

}

std::cout << "\n";

}

}

```

```

// Print year labels centered below the graph

std::cout << "    "; // Space before year labels

for (const auto& c : data) {

    std::cout << std::setw(6) << c.year;

}

std::cout << "\n";

}

```

```

// Function to display candlestick data in a tabular format

void displayCandlestickData(const std::vector<Candlestick>& data) {

    if (data.empty()) {

        std::cout << "No data to display.\n";

        return;

    }

```

```

// Print table header

std::cout << std::setw(8) << "Year" << std::setw(12) << "Open"

    << std::setw(12) << "Close" << std::setw(12) << "High"

    << std::setw(12) << "Low" << "\n";

std::cout << std::string(56, '-') << "\n";

```

```

// Print data rows
for (const auto& c : data) {
    std::cout << std::setw(8) << c.year
        << std::setw(12) << std::fixed << std::setprecision(2) << c.open
        << std::setw(12) << std::fixed << std::setprecision(2) << c.close
        << std::setw(12) << std::fixed << std::setprecision(2) << c.high
        << std::setw(12) << std::fixed << std::setprecision(2) << c.low
        << "\n";
}
}

int main() {
    // Path to the dataset
    std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";
    std::string country;

    std::cout << "Enter the countrycode_temperature (e.g., GB_temperature): ";
    std::cin >> country;

    // Read and process the file for the selected country
    std::vector<Candlestick> candlesticks = readAndProcessFile(filePath, country);

    if (candlesticks.empty()) {
        std::cerr << "No data available for the selected country.\n";
        return 1;
    }
}

```



```

std::vector<Candlestick> filteredData = candlesticks;

while (true) {
    // Display filter options

    std::cout << "\nFilter Options:\n";

    std::cout << "1. Filter by Date Range\n";

    std::cout << "2. Filter by Temperature Range\n";

    std::cout << "3. Reset Filters\n";

    std::cout << "4. Plot Data\n";

    std::cout << "5. Display Data\n";

    std::cout << "6. Exit\n";

    std::cout << "Enter your choice: ";

    int choice;

    std::cin >> choice;

    if (choice == 1) {
        int startYear, endYear;

        std::cout << "Enter start year: ";

        std::cin >> startYear;

        std::cout << "Enter end year: ";

        std::cin >> endYear;

        filteredData = filterByDateRange(filteredData, startYear, endYear);

        std::cout << "Data filtered by date range.\n";
    } else if (choice == 2) {
        double minTemp, maxTemp;

        std::cout << "Enter minimum temperature: ";

        std::cin >> minTemp;
    }
}

```

```

std::cout << "Enter maximum temperature: ";

std::cin >> maxTemp;

filteredData = filterByTemperatureRange(filteredData, minTemp, maxTemp);

std::cout << "Data filtered by temperature range.\n";

} else if (choice == 3) {

    filteredData = candlesticks; // Reset filters

    std::cout << "Filters reset.\n";

} else if (choice == 4) {

    // Check if the year range exceeds 10 years

    if (!filteredData.empty()) {

        int startYear = filteredData.front().year;

        int endYear = filteredData.back().year;

        if (endYear - startYear > 10) {

            std::cout << "Cannot display: Year range exceeds 10 years.\n";

        } else {

            plotTextGraph(filteredData);

        }

    } else {

        std::cout << "Cannot display: No data available for the current filters.\n";

    }

} else if (choice == 5) {

    displayCandlestickData(filteredData); // Display data in aligned format

} else if (choice == 6) {

    std::cout << "Exiting.\n";

    break;

} else {

    std::cout << "Invalid choice. Please try again.\n";

```

```
    }  
}  
  
    return 0;  
}
```

//Task 4

```
#include <iostream>  
  
#include <vector>  
  
#include <cmath>  
  
#include <iomanip>  
  
#include <fstream>  
  
#include <sstream>  
  
#include <string>  
  
#include <map>  
  
#include <algorithm>  
  
#include <numeric>  
  
  
// Function to safely convert a string to double  
double safeStringToDouble(const std::string& str) {  
    try {  
        if (str.empty()) throw std::invalid_argument("Empty string");  
        return std::stod(str);  
    } catch (...) {  
        return std::numeric_limits<double>::quiet_NaN();  
    }  
}
```

```

// Function to read and process the file for a specific country

std::map<int, double> readDataForCountry(const std::string& filePath, const
std::string& country) {

    std::ifstream file(filePath);

    if (!file.is_open()) {

        std::cerr << "Error: Unable to open file: " << filePath << std::endl;

        return {};

    }

    std::string line;

    std::map<int, std::vector<double>> yearlyTemps;

    // Parse the header row to find the column for the specified country

    std::getline(file, line);

    std::stringstream headerStream(line);

    std::string columnName;

    int countryColumnIndex = -1;

    int currentIndex = 0;

    while (std::getline(headerStream, columnName, ',')) {

        if (columnName == country) {

            countryColumnIndex = currentIndex;

            break;

        }

        currentIndex++;

    }

    if (countryColumnIndex == -1) {

```

```

std::cerr << "Error: Country not found in the dataset.\n";

return {};
}

// Process the data for the selected country
while (std::getline(file, line)) {
    std::stringstream lineStream(line);

    std::string date, tempStr;

    std::getline(lineStream, date, ',');

    int year = std::stoi(date.substr(0, 4));

    for (int i = 0; i <= countryColumnIndex; ++i) {
        std::getline(lineStream, tempStr, ',');
    }

    double temperature = safeStringToDouble(tempStr);

    if (!std::isnan(temperature)) {
        yearlyTemps[year].push_back(temperature);
    }
}

// Calculate the average temperature per year
std::map<int, double> avgTemps;

for (const auto& entry : yearlyTemps) {
    const auto& temps = entry.second;

    avgTemps[entry.first] = std::accumulate(temps.begin(), temps.end(), 0.0) /
temps.size();
}

```

```
    return avgTemps;
}
```

```
// Function to compute polynomial regression coefficients
```

```
std::vector<double> fitPolynomial(const std::vector<int>& x, const
std::vector<double>& y, int degree) {
```

```
    int n = x.size();
```

```
    int d = degree + 1;
```

```
    // Initialize matrices
```

```
    std::vector<std::vector<double>> X(d, std::vector<double>(d, 0));
```

```
    std::vector<double> Y(d, 0);
```

```
    // Compute X matrix (sum of powers of x)
```

```
    for (int i = 0; i < d; ++i) {
```

```
        for (int j = 0; j < d; ++j) {
```

```
            for (int k = 0; k < n; ++k) {
```

```
                X[i][j] += std::pow(x[k], i + j);
```

```
            }
```

```
        }
```

```
    }
```

```
    // Compute Y vector (sum of  $x^i * y$ )
```

```
    for (int i = 0; i < d; ++i) {
```

```
        for (int k = 0; k < n; ++k) {
```

```
            Y[i] += std::pow(x[k], i) * y[k];
```

```
        }
```

```
}
```

```
// Solve for coefficients using Gaussian elimination
```

```
std::vector<double> coefficients(d, 0);
```

```
for (int i = 0; i < d; ++i) {
```

```
    // Normalize the pivot row
```

```
    double pivot = X[i][i];
```

```
    for (int j = 0; j < d; ++j) {
```

```
        X[i][j] /= pivot;
```

```
    }
```

```
    Y[i] /= pivot;
```

```
// Eliminate other rows
```

```
for (int k = 0; k < d; ++k) {
```

```
    if (k != i) {
```

```
        double factor = X[k][i];
```

```
        for (int j = 0; j < d; ++j) {
```

```
            X[k][j] -= factor * X[i][j];
```

```
        }
```

```
        Y[k] -= factor * Y[i];
```

```
    }
```

```
}
```

```
}
```

```
// Extract coefficients
```

```
for (int i = 0; i < d; ++i) {
```

```
    coefficients[i] = Y[i];
```

```
}
```

```
    return coefficients;
}
```

```
// Function to predict temperature using polynomial regression
```

```
double predictPolynomial(const std::vector<double>& coefficients, int x) {
    double prediction = 0.0;
    for (size_t i = 0; i < coefficients.size(); ++i) {
        prediction += coefficients[i] * std::pow(x, i);
    }
    return prediction;
}
```

```
// Function to plot the predictions
```

```
void plotPredictions(const std::vector<std::pair<int, double>>& predictions) {
    if (predictions.empty()) {
        std::cout << "No predictions to plot.\n";
        return;
    }
}
```

```
// Determine min and max temperature and round to nearest multiples of 0.05
```

```
double minTemp = std::numeric_limits<double>::max();
double maxTemp = std::numeric_limits<double>::lowest();
for (const auto& p : predictions) {
    minTemp = std::min(minTemp, p.second);
    maxTemp = std::max(maxTemp, p.second);
}
double roundedMinTemp = std::floor(minTemp / 0.05) * 0.05;
```



```

double roundedMaxTemp = std::ceil(maxTemp / 0.05) * 0.05;

// Ensure a consistent tick interval of 0.05
double tickInterval = 0.05;

// Calculate the number of rows for the y-axis
int numTicks = static_cast<int>((roundedMaxTemp - roundedMinTemp) / tickInterval) +
1;

// Create a grid for the plot
const int columnWidth = 8;
const int plotHeight = numTicks - 1;

for (int row = plotHeight; row >= 0; --row) {
    double currentValue = roundedMinTemp + row * tickInterval;

    // Print y-axis label
    std::cout << std::setw(6) << std::fixed << std::setprecision(2) << currentValue << " |";

    // Plot the points for each year
    for (const auto& p : predictions) {
        // Calculate the row for this prediction
        int predictedRow = static_cast<int>((p.second - roundedMinTemp) / tickInterval +
0.5);

        // Prepare column for alignment
        std::string column(columnWidth, ' ');
        if (row == predictedRow) {
            column[columnWidth / 2] = '*'; // Place the '*' in the middle

```

```

    }

    std::cout << column;

    }

    std::cout << "\n";

}

// Print the x-axis (year labels)

std::cout << "    "; // Offset for y-axis labels

for (const auto& p : predictions) {

    std::cout << std::setw(columnWidth) << p.first;

}

std::cout << "\n";

}

int main() {

    std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";

    std::string country;

    std::cout << "Enter the countrycode_temperature (e.g., GB_temperature): ";

    std::cin >> country;

    auto avgTemps = readDataForCountry(filePath, country);

    if (avgTemps.empty()) {

        std::cerr << "No data available for the selected country.\n";

        return 1;

    }

```

```

std::vector<int> years;
std::vector<double> temperatures;
for (const auto& entry : avgTemps) {
    years.push_back(entry.first);
    temperatures.push_back(entry.second);
}

int degree = 2; // Degree of the polynomial
auto coefficients = fitPolynomial(years, temperatures, degree);

int startYear, endYear;
std::cout << "Enter start year for prediction: ";
std::cin >> startYear;
std::cout << "Enter end year for prediction: ";
std::cin >> endYear;

// Check if the year range exceeds 10
if (endYear - startYear > 10) {
    std::cout << "Year range exceeds 10 years.\n";
    return 0;
}

if (startYear <= 2019) {
    std::cout << "Cannot predict: Year range includes 2019 or earlier.\n";
    return 0;
}

// Perform predictions if range is valid

```

```
std::vector<std::pair<int, double>> predictions;

for (int year = startYear; year <= endYear; ++year) {
    double predictedTemp = predictPolynomial(coefficients, year);
    predictions.emplace_back(year, predictedTemp);
}

std::cout << "\nPredicted Temperatures:\n";
for (const auto& p : predictions) {
    std::cout << "Year: " << p.first << ", Predicted Avg Temp: " << p.second << "°C\n";
}

std::cout << "\nPrediction Plot:\n";
plotPredictions(predictions);

return 0;
}

//end
```